

SENTINEL

A DISTRIBUTED BORDER SURVEILLANCE SYSTEM WITH DUPLICATE DETECTION AND SECURE ALERT PROCESSING

Detailed Technical Project Report

Guided by

Dr. S.K. Saravanan

Presented by

B.V. Tharun — Regn. No. 192525155

D. Dinesh Reddy — Regn. No. 192525186

PROJECT FOCUS Duplicate-safe distributed object detection using POSIX threads, TCP/IP sockets, dynamic memory, recursive zone search and mutex-protected, IPC-driven alerting.

Executive Snapshot

Sentinel is a distributed border-monitoring engine built to solve a very concrete systems problem: multiple independent border stations can detect the same physical object and report it separately, causing the system to double-count real-world objects. The project combines multithreaded feed ingestion, socket-based data transport, dynamic record management, similarity-based duplicate detection, recursive zone lookup and mutex-protected database synchronization into one working surveillance pipeline, with file-based logging and IPC alerting closing the loop from detection to response.

Dimension	Sentinel Approach
Concurrency	POSIX threads, one worker per station feed
Communication	TCP/IP client–server sockets (station → command center)
Data Handling	Dynamically allocated object records via struct pointers and malloc()
Correctness	Duplicate detection & merge, matched on ID + location + time
Zone Resolution	Recursive parent → child border-zone search
Synchronization	Mutex-protected updates to the central object database
Alerting	IPC-based alert hand-off to an independent response process
Persistence / Audit	File-based event log (surveillance.log)

01 Project Overview & Problem Definition

Abstract

Sentinel is a C-based distributed surveillance engine that models a border region monitored by multiple independent detection stations. Each station observes objects entering its field of view and reports them to a central command center. The key engineering challenge is that two or more stations may detect the same physical object within a short time window and report it as if it were two separate objects. Sentinel addresses this using POSIX threads for concurrent feed ingestion, TCP/IP sockets for transport, dynamically allocated object records, an ID + location + time duplicate-matching engine, recursive hierarchical zone search, and mutex-protected updates to a shared central database, with every event logged to file and forwarded through an IPC alert channel to a response process.

1. The Duplicate Detection Problem

A border object is a real-world entity that can legitimately fall within the sensing range of more than one station. When Station A and Station B each detect the same object at nearly the same time, both stations independently transmit a detection record to the command center. Without correlation logic, the command center has no way of knowing that the two incoming records describe one object rather than two.

DUPLICATE DETECTION PROBLEM

STATION A detects object → reports (ID, location, t_1)

STATION B detects object → reports (ID, location, t_2)

same object · nearly the same time → **TWO REPORTS, ONE REAL OBJECT**

Without correlation, the command center commits both reports as independent records.

CORE FAILURE A single physical object detected by two stations within the match window is logged as two independent records unless the reports are correlated by ID, location and time before either is committed to the central database.

Impact

- Multiple stations can legitimately observe the same real-world object.
- Independent detections can reach the command center before they are correlated.
- The tracked object count can diverge from the true number of objects at the border.
- Whether a duplicate is actually created depends on network timing and feed arrival order.

02 Objectives, Architecture & Technology Stack

2. Project Objectives

- Detect and receive live object data from multiple distributed border stations.
- Ingest multiple concurrent station feeds using POSIX threads without one feed blocking another.
- Transmit detection data reliably from station to command center using TCP/IP sockets.
- Dynamically register incoming object records using pointer-based structures and malloc().
- Classify and de-duplicate detections by matching ID, location and time.
- Recursively traverse hierarchical border zones to locate the correct database segment for each object.
- Protect the central object database from concurrent write conflicts using mutex locks.
- Log every surveillance event and issue alerts through an inter-process communication (IPC) channel.

3. Architecture

Layer	Components	Responsibility
Sensing / Station Layer	Station A / B / C detection feeds	Detect border objects and report them
Transport	TCP/IP sockets	Move detection data to the command center
Concurrency	pthreads (one worker per feed)	Parallel ingestion of station feeds
Data Registration	struct pointers, malloc()	Dynamic creation of object records
Deduplication Engine	ID + location + time matcher	Merge duplicate detections into one record
Zone Management	Recursive zone traversal	Locate the correct hierarchical database zone
Persistence / Sync	Mutex-protected central object database	Consistent, conflict-free record updates
Alerting	IPC channel, response process	Notify and trigger the response workflow
Audit	surveillance.log	Ordered, file-based historical event record

4. Technology Stack

Area	Technology	Role
Language	C (C11 / GNU11)	Core detection and processing engine
Threads	POSIX pthreads	Concurrent station-feed worker threads
Synchronization	pthread_mutex_t	Protect the shared central object database
Networking	TCP/IP sockets	Station-to-command-center transport
Memory Management	malloc(), struct pointers	Dynamic creation of object records

Area	Technology	Role
Data Structures	Central object list / linked records	Track live and merged detections
Recursion	Recursive C functions	Hierarchical border-zone traversal
IPC	Inter-process communication	Alert delivery to the response process
Logging	File I/O (surveillance.log)	Persistent, ordered audit trail
Environment	Linux, GCC, Make	Build and execution environment

DESIGN PRINCIPLE Never trust a single detection in isolation — correlate every incoming report against recent history before it is allowed to become a new record.

03 Concurrent Feed Ingestion & Duplicate Detection

5. Parallel Station Feed Ingestion

Sentinel uses POSIX threads to ingest live data from multiple border stations at the same time. A dedicated worker thread is created for each station feed connection, so a slow or bursty feed from one station cannot block the ingestion of data from another. Each worker reads incoming detection packets from its own socket and hands them to the registration stage independently of every other worker.

PARALLEL FEED INGESTION

STATION A FEED → pthread worker

STATION B FEED → pthread worker → **COMMAND CENTER**

STATION C FEED → pthread worker

Feeds are ingested independently — unrelated stations never wait on each other.

Feed Worker Startup

```
for (i = 0; i < station_count; i++) {
    pthread_create(&worker[i], NULL,
                  station_feed_worker, &stations[i]);
}
```

Without Threads	With Pthread Workers
One feed connection blocks all ingestion.	Each station feed is read on its own thread.
A late or slow station delays every station.	A slow feed only delays its own worker.
Throughput is capped by the slowest station.	Throughput follows the fastest available feeds.
Correctness can depend on feed arrival order.	Feed arrival order no longer decides correctness.

6. Duplicate Detection & Merge Logic

Every registered object record is compared against recently registered records from other stations using three fields: object ID (when a sensor supplies one), approximate location, and detection time. When a new report falls within the configured match window on all three fields, Sentinel treats it as the same physical object and merges it into the existing record instead of creating a new one.

RULE Two detection reports are merged into a single record only when ID, location and time all fall inside the configured match window; a report that fails even one of these checks becomes a new record.

Why Match on Three Fields?

Matching on ID alone is unreliable because not every sensor reports a stable object ID. Matching on location or time alone is unreliable because unrelated objects can appear in the same zone or at the same moment. Combining all three keeps false merges rare while still catching genuine same-object duplicates reliably.

04 Networking & Dynamic Object Registration

7. TCP/IP Socket Communication

Sentinel uses TCP sockets to separate each border station from the central command-center process. This models real detection hardware or edge sensors reporting to a monitoring service over a network. TCP provides an ordered, reliable byte stream, so detection packets arrive at the command center in the order each station sent them, without silent loss or corruption.

8. Dynamic Object Registration

As each detection arrives over the socket, the command center allocates a new object record on the heap using `malloc()` and populates it through a struct pointer. Dynamic allocation lets the central object list grow only as real detections arrive, with no fixed upper bound compiled into the program, and lets a record be freed once it is merged, expired or archived.

OBJECT REGISTRATION

STATION DATA → SOCKET (TCP/IP) → REGISTER OBJECT

struct pointer + malloc()



CENTRAL OBJECT LIST

Sockets transfer the data; dynamic memory creates a record for every object as it arrives.

05 Zone Synchronization, Logging & Alerting

9. Recursive Zone Search & Mutex-Protected Updates

The border region is organized as a hierarchy of zones — Zone A, Zone B and Zone C, each subdivided into sub-zones such as A1/A2, B1/B2 and C1/C2. Once an object record is ready to be filed, a recursive function walks the hierarchy from the parent zone down to the matching child zone, following the object's reported coordinates at each level until it reaches the correct leaf zone.

The recursive search itself touches no shared state and can run freely. The moment the correct leaf zone is found, its record set is updated under a mutex lock, so two objects being filed into the same zone at the same time cannot corrupt each other's write.

SAFE DB UPDATE A zone segment is never modified outside its mutex: the recursive search runs unlocked, but the instant a matching leaf zone is found, its lock is acquired before any write and released immediately after.

Zone Hierarchy & Synchronization Features

Feature	Purpose	Demonstrates
Recursive Zone Search	Walk parent zone → child zone to find the exact border segment	Hierarchical problem decomposition
Mutex-Protected Update	Serialize writes to a located zone's records	Safe concurrent database access
Struct-Pointer Registration	Create object records dynamically via malloc()	Pointer-based dynamic memory management
Duplicate Merge Engine	Combine near-simultaneous reports of one object	Correctness under concurrent input
IPC Alert Dispatch	Hand confirmed detections to the response process	Inter-process communication

Suggested Operational Sequence

- Start all station feeds reporting into the command center.
- Trigger two near-simultaneous detections of the same object from Station A and Station B.
- Observe the duplicate-check logic merge the two reports into a single record.
- Trigger a detection inside a nested zone (e.g. Zone B → B2) and confirm the recursive search resolves it.
- Run concurrent detections across different zones and confirm mutex-protected updates stay consistent.
- Inspect surveillance.log to confirm every event was recorded and an alert was raised.

10. Surveillance Log (File-Based Audit Trail)

Sentinel maintains a plain-text surveillance log, `surveillance.log`, that records every detection, merge, zone assignment and alert in the order it happened. The log gives investigators a durable, human-readable history of what the system observed and how each object record was resolved, independent of the live, in-memory database.

11. IPC-Based Alerting & End-to-End Detection Lifecycle

Once an object record is committed — whether newly created or merged — the command center sends an alert through an inter-process communication channel to a separate response process. Decoupling detection from response lets the response workflow (dispatch, notification, escalation) evolve independently of the detection engine.

Property	Mechanism	Result
Auditability	Ordered entries in <code>surveillance.log</code>	Traceable, replayable event history
Timely Response	IPC alert channel to a response process	Rapid hand-off from detection to action
Consistency	Mutex-protected writes committed before logging	The log always reflects committed state
Single Source of Truth	One central, zone-organized object database	One record per real-world object

TECHNICAL PRECISION IPC alerting notifies the response process promptly, but is not by itself a guarantee of physical interception — it hands off a verified detection to a human or automated response workflow.

End-to-End Detection Lifecycle

```
Station Detects Object
↓
TCP Socket → Command Center
↓
Pthread Worker Receives Feed
↓
Register Object (struct pointer + malloc())
↓
Duplicate Check (ID + Location + Time)
↓
├─ Match      → Merge into Existing Record
└─ No Match   → Create New Record
↓
Recursive Zone Search
↓
Mutex Lock → Update Central DB → Unlock
↓
Log Event (surveillance.log)
↓
IPC Alert → Response Process
```


06 Testing, Results, Limitations & Conclusion

12. Validation Strategy

Test	Expected Safe Behavior	Purpose
Duplicate Report Test	The same object is logged exactly once	Deduplication correctness
Concurrent Multi-Station Feed	All feeds are processed without loss or blocking	Threading correctness
Nested Zone Traversal Test	The correct hierarchical zone is located	Recursion correctness
Concurrent DB Write Test	No lost or conflicting zone updates	Mutex protection
Alert Delivery Test	The response process is notified for every valid detection	IPC correctness
Log Integrity Check	A complete, ordered event history is recoverable	Auditability

13. Key Outcomes

- The project demonstrates low-level distributed-systems programming in C rather than hiding concurrency and networking behind a framework.
- The duplicate-detection failure mode is intentionally reproducible, making the correctness improvement observable end to end.
- Per-feed pthread workers preserve ingestion throughput while the duplicate-matching engine keeps the object count accurate.
- Recursive zone traversal gives a simple, explainable way to route each object to the correct hierarchical database segment.
- Mutex-protected zone updates keep the central database consistent without serializing the whole system.
- The surveillance log and IPC alert channel extend the design from in-memory correctness to a durable, actionable audit trail.

14. Limitations & Future Enhancements

- A deployed border-monitoring platform would require authenticated stations and encrypted transport such as TLS.
- The prototype log would need hardened storage and crash-recovery semantics for continuous field operation.
- ID + location + time matching detects near-simultaneous duplicates but does not perform full sensor fusion or object tracking across time.
- Production deployment would require redundancy across command centers and extensive fault-injection testing.
- Future work could add TLS-secured sockets, GPS/sensor fusion, a persistent database backend, and a live monitoring dashboard.

15. Conclusion

SENTINEL IN ONE SENTENCE Sentinel demonstrates that a distributed sensor network can stay both responsive and accurate by combining concurrent feed ingestion, deterministic duplicate matching, recursive zone resolution and mutex-protected, IPC-driven alerting.

Sentinel presents C as a systems-engineering tool for reasoning about threads, sockets, dynamic memory, recursion and synchronization in a single distributed pipeline. The central lesson mirrors any multi-source monitoring system: raw detection throughput is easy to obtain with concurrency, while trustworthy counts come only from deliberately correlating and protecting the state that must remain consistent.

Keywords: C11 • GNU11 • POSIX Threads • pthread_mutex_t • TCP/IP Sockets • Client–Server • malloc() • Struct Pointers • Recursion • Duplicate Detection • IPC • Surveillance Logging • Distributed Systems